Individual Project Proposal — Assignment 3

Course: CSC10014 – Computational Thinking
Semester: Fall 2026–2027 | **Class:** 25A01
Student: Tran Duc Toan — 25125091
Date: October 16, 2026

---

1. Project Title

UrbanPulse: A Context-Aware Conversational Routing Assistant for Real-Time Urban Travel Optimization

---

2. Relationship to Group Project

This proposal **builds upon** our group's project — *Context-Aware Dynamic Routing Assistant Minimizing Real-World Urban Travel Time* — but pivots it into a **conversational Smart Virtual Assistant** that meets all the project requirements outlined in the 2026 project specification (Engineering a Smart Virtual Assistant).

Aspect	Group Proposal	This Individual Proposal
Core framing	A routing engine that computes optimal paths over a time-dependent weighted graph $G = (V, E, W(t))$	A conversational assistant that wraps the routing engine inside a full NLU → Routing → Retrieval → Reasoning → Verification → Response pipeline
User interaction	Implied map UI / API call	Natural-language chat interface (Gradio / Streamlit) where users describe trips in plain language
Problem domain	Purely navigation (shortest path)	Navigation **+** urban situational awareness (incidents, weather, flooding, road closures, community reports) via a multi-source knowledge retrieval layer
Key CT difference	Graph algorithm optimization	Full computational pipeline design: intent classification, deterministic rule engine for constraint validation, probabilistic reasoning for risk scoring, and AI-assisted community report extraction — all orchestrated by an explicit router

> **Why the difference is meaningful:** The group project focuses on *algorithmic optimization within a graph*. This proposal instead foregrounds the *system design and orchestration problem* — deciding *when* to use the graph algorithm, *when* to query live APIs, *when* to invoke an LLM for natural-language understanding, and *when* to fall back to a deterministic rule. This is the heart of Computational Thinking applied to virtual assistant engineering.

---

3. Problem Definition & Context Analysis

3.1 The Problem

Urban commuters in Ho Chi Minh City face unpredictable travel times due to:

- **Dynamic congestion** that shifts by the minute during rush hours.
- **Hyper-local incidents** (accidents, flooding during rainy season, road construction, police checkpoints) that are reported on social media and community forums (e.g., Facebook traffic groups, Zalo channels) *before* they appear on Google Maps.
- **Multi-modal decisions**: Should I take a motorbike through the alley shortcut? Switch to the metro? Wait for the rain to stop?

Current navigation tools (Google Maps, Grab) provide route suggestions but **do not let the user have a conversation** about their constraints, preferences, and real-time situation.

3.2 Target User

University students and young professionals in HCMC who commute by motorbike or public transit and want to *talk to* an assistant about their trip rather than just type in an address.

3.3 Representative Use Cases (8–12)

#	User Request (Natural Language)	Expected Computational Behavior
1	*"I need to get from Thủ Đức to District 1 by 8 AM, and it's raining."*	Parse intent → extract origin/destination/deadline/weather constraint → query weather API → apply rain penalty weights on graph edges → run Time-Dependent A* → return route with ETA
2	*"Is Điện Biên Phủ flooded right now?"*	Classify as **information retrieval** → query community incident database → retrieve recent reports → summarize → respond
3	*"Find me a route that avoids highway, I'm on a bicycle."*	Parse constraint (no highway, vehicle=bicycle) → modify graph edge filter → compute route → respond

#	User Request (Natural Language)	Expected Computational Behavior
4	*"Compare motorbike vs. bus for my trip to Bình Thạnh."*	Spawn two route computations (motorbike graph, bus transit graph) → compare ETA, cost, risk → present structured comparison
5	*"I heard there's an accident on Nguyễn Hữu Cảnh, is that true?"*	Classify as **verification query** → search incident DB + live API → cross-reference → confidence score → respond with evidence
6	*"Save this route as my daily commute."*	Classify as **action/save** → store route profile → confirm
7	*"What's the safest time to leave for Tân Sơn Nhất airport tomorrow?"*	Classify as **planning** → historical congestion analysis for each departure window → risk scoring → recommend optimal departure time
8	*"Tell me a joke."*	Classify as **out-of-scope** → polite refusal / redirect → "I'm a routing assistant, I can help you plan your trip!"
9	*"Nên đi đường nào từ quận 7 lên quận 3 giờ tan tầm?"* (Vietnamese)	Language detection → Vietnamese NLU → same routing pipeline → respond in Vietnamese
10	*"Add a report: road blocked on Lý Thường Kiệt near Tân Bình."*	Classify as **community report submission** → validate location → geocode → store in incident database → acknowledge

3.4 Non-Goals

- **Not** a general chatbot — will refuse non-routing, non-traffic queries.
- **Not** a real-time GPS tracker — provides route recommendations, not turn-by-turn live tracking.
- **Not** a ride-hailing service — does not book rides.

3.5 Success Criteria (Measurable)

Metric	Target
Intent classification accuracy (on a 50-case test set)	≥ 85%
Route ETA error vs. Google Maps baseline	≤ 15% relative error
Incident retrieval precision (are reported incidents real?)	≥ 70%
End-to-end response latency	≤ 5 seconds for routing queries

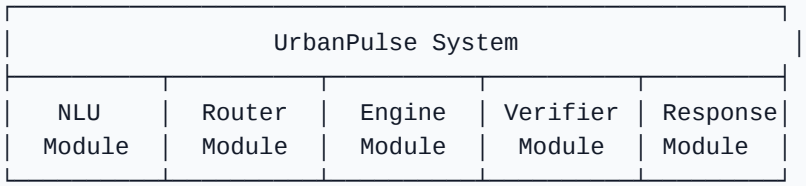
Metric	Target
Fallback rate (queries correctly refused or escalated)	≥ 90% for out-of-scope inputs

---

4. Applying the Four Pillars of Computational Thinking

4.1 Decomposition

The system is decomposed into **five modules**, each independently testable:



1. **NLU Module** — Understands the user's natural-language request.
    - Sub-decomposition: Language detection → Intent classification → Entity extraction (origin, destination, time, vehicle, constraints).
2. **Router Module** — Deterministic decision tree that routes the parsed request to the correct computational engine.
    - Sub-decomposition: Intent-to-capability mapping → Constraint validation → Fallback/clarification logic.
3. **Engine Module** — The computational workhorse, containing three sub-engines:
    - **Graph Routing Engine**: Time-dependent weighted graph algorithms (Dijkstra / A*).
    - **Retrieval Engine**: Queries community incident database, weather API, OpenStreetMap API.
    - **AI Reasoning Engine**: LLM-based summarization, community report parsing, comparison generation.
4. **Verification Module** — Validates outputs before presenting to the user.
    - Sub-decomposition: Constraint-satisfaction check → ETA sanity check → Source confidence scoring.
5. **Response Module** — Formats and delivers the answer.
    - Sub-decomposition: Template selection → Natural-language generation → Citation/evidence attachment.

4.2 Pattern Recognition

Pattern Identified	Where It Appears	How It Is Exploited
Recurring congestion patterns	Historical traffic data by time-of-day and day-of-week	Edge weights $W(e, t)$ are modeled as periodic functions; cluster similar time slots to predict future congestion
Incident report structure	Community posts follow patterns: `[Location] + [Incident Type] + [Time/Status]`	NLP extraction using regex + LLM classification; template matching identifies report type
User intent patterns	Users phrase routing requests in predictable templates ("How to get from X to Y", "Is X flooded?")	Few-shot intent classifier trained on labeled examples
Seasonal weather patterns	HCMC flooding is seasonal (May–November)	Historical flood-risk map overlaid on graph edges as a seasonal penalty weight

4.3 Abstraction

Abstraction Layer	What It Hides	What Interface It Exposes
Graph Abstraction	Raw OSM road data, GPS coordinates, road segments	`Graph.shortest_path(origin, destination, time, constraints) → Route`
Data Source Abstraction	HTTP calls to OpenWeather, OSM, community DB	`DataProvider.query(source_type, params) → StructuredData`
AI Abstraction	Specific LLM model (Gemini, Ollama, etc.)	`AIService.classify(text) → Intent`, `AIService.extract(text) → Entities`, `AIService.summarize(text) → Summary`
User Interface Abstraction	Gradio/Streamlit widgets	`ChatInterface.send(message) → Response`

Key abstraction decision: The routing engine does not know whether it was triggered by a Vietnamese or English query, whether the user is on mobile or desktop, or whether the weather data came from OpenWeather or a cached local file. This separation of concerns makes each module independently testable.

4.4 Algorithm Design

Core Algorithm: Context-Aware Time-Dependent A*

```
FUNCTION ContextAwareRoute(G, source, target, departure_time, constraints):
    // Phase 1: Graph Preparation
    G' ← FilterGraph(G, constraints)          // Remove edges violating constraints
    W  ← LoadDynamicWeights(G', departure_time) // Time-dependent edge weights
    R  ← LoadRiskOverlay(departure_time)        // Incident + weather penalties

    // Phase 2: Weighted A* Search
    FOR each edge e in G':
        cost(e) ← W(e, t_current) + α·R(e) + β·elevation(e)

    path ← A_Star(G', source, target, cost, heuristic=haversine)

    // Phase 3: Verification
    IF EstimatedTime(path) > constraints.deadline:
        RETURN FallbackRoute(G, source, target, constraints)  // Greedy fallback

    RETURN path, EstimatedTime(path), RiskScore(path)
```

Parameters:

- α: risk sensitivity weight (tunable per user preference)
- β: elevation penalty weight (relevant for bicycles/motorbikes in rain)

Intent Classification Algorithm (Deterministic + AI Hybrid)

```
FUNCTION ClassifyAndRoute(user_message):
    // Step 1: Rule-based fast path (deterministic)
    IF matches_pattern(user_message, GREETING_PATTERNS):
        RETURN Intent.GREETING
    IF matches_pattern(user_message, OUT_OF_SCOPE_PATTERNS):
        RETURN Intent.OUT_OF_SCOPE

    // Step 2: AI classification (for ambiguous queries)
    intent, confidence ← LLM.classify(user_message, few_shot_examples)

    IF confidence < THRESHOLD:
        RETURN Intent.CLARIFICATION_NEEDED

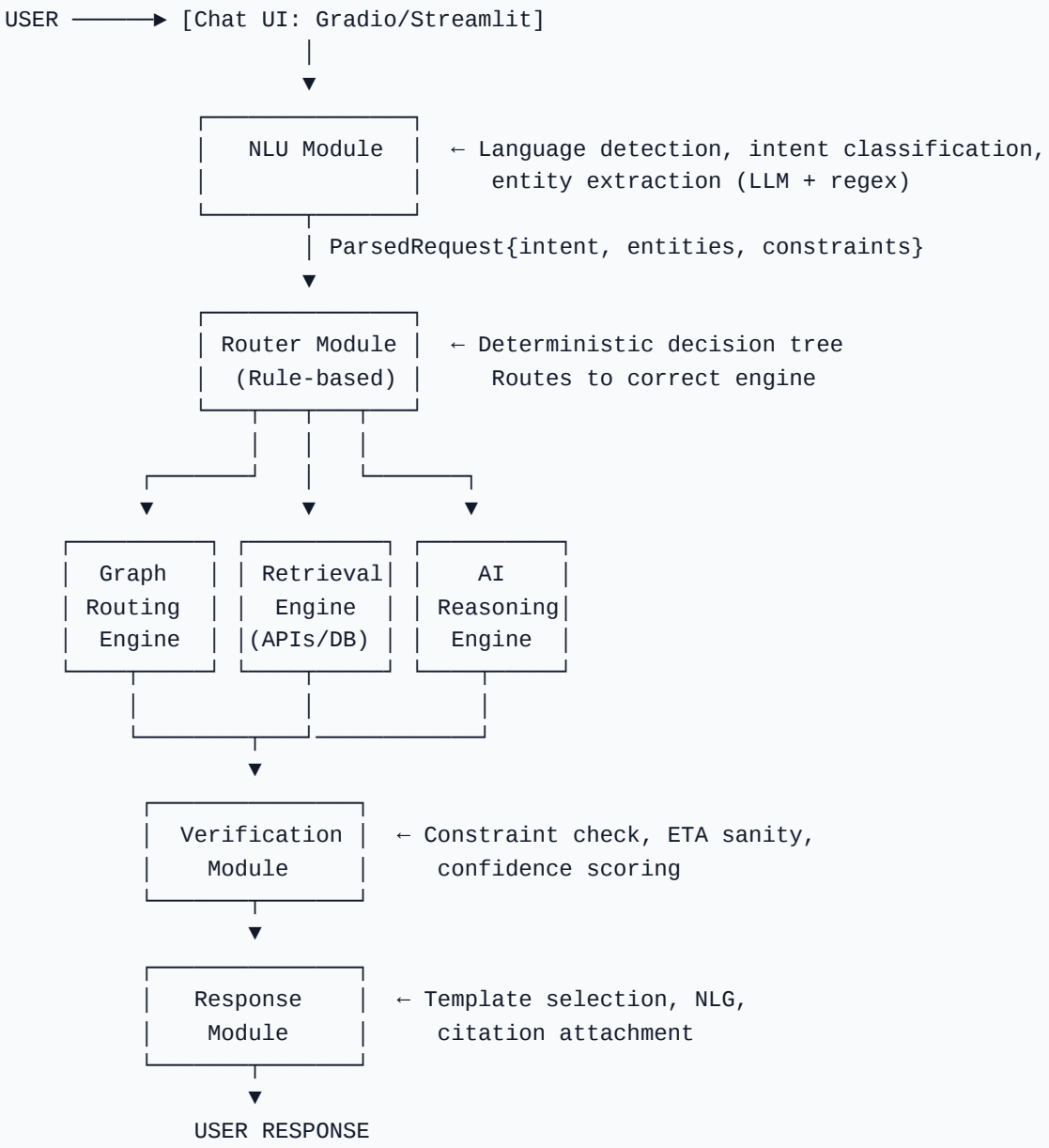
    // Step 3: Entity extraction
    entities ← LLM.extract_entities(user_message)
    validated_entities ← validate_and_geocode(entities)

    RETURN intent, validated_entities
```

Why this hybrid approach? Pure rule-based is too brittle for natural language. Pure LLM is too slow and non-deterministic for simple patterns. The hybrid captures the best of both — a key CT design decision.

---

5. System Architecture



Technology Stack

Component	Technology	Justification
Chat UI	Gradio or Streamlit	Rapid prototyping (Lab 3 skill: Gradio/Streamlit)
NLU	Regex + Ollama / Gemini API (few-shot)	Hybrid deterministic + AI classification
Graph Routing	NetworkX + OSMnx (Python)	Provides OSM graph data + shortest-path algorithms
Weather API	OpenWeather API	Real-time weather for HCMC (Lab 6 skill)

Component	Technology	Justification
Map Data	OpenStreetMap API / OSMnx	Street network extraction (Lab 6 skill)
Incident DB	JSON/JSONL file or Firebase	Structured community incident storage (Lab 3 + Lab 8 skills)
Backend API	FastAPI	Modular API endpoints (Lab 5 skill)
Testing	pytest + custom benchmark suite	Unit + integration testing (Lab 9 skill)

---

6. Data & Knowledge Design

6.1 Structured Data Sources

Source	Format	Usage
OpenStreetMap (via OSMnx)	Graph $G = (V, E)$	Road network topology
OpenWeather API	JSON	Real-time weather conditions → rain/flood risk weights
Historical congestion data	CSV/JSONL	Time-of-day congestion patterns → edge weight functions $W(e, t)$
Community incident database	JSONL	User-submitted and NLP-extracted road incidents

6.2 Unstructured Data Sources

Source	Processing	Output
Community traffic reports (text)	LLM entity extraction (location, incident type, severity, time)	Structured incident record in JSONL
User chat messages	Intent classification + entity extraction	ParsedRequest object

6.3 Data Schema: Incident Record

```
{
  "id": "INC-20261016-001",
  "location": {"lat": 10.7769, "lon": 106.7009, "address": "Nguyễn Hữu Cảnh, Bình
Thạnh"},
  "type": "flooding",
  "severity": "moderate",
  "source": "community_report",
  "confidence": 0.82,
  "reported_at": "2026-10-16T07:30:00+07:00",
  "expires_at": "2026-10-16T12:00:00+07:00",
  "raw_text": "Nguyễn Hữu Cảnh ngập nặng, nước tới đầu gối"
}
```

---

7. Meeting the 8 Minimum System Requirements

ID	Requirement	How UrbanPulse Satisfies It
R1	Problem boundary	Defined target user (HCMC commuters), 10 use cases, explicit non-goals, 5 measurable success criteria
R2	Deterministic capability	Rule-based intent router; constraint validation (deadline check, vehicle filter); graph algorithm (A*) is fully transparent and testable
R3	Data / retrieval capability	OpenStreetMap graph data, OpenWeather API, community incident JSONL database — all with documented provenance
R4	AI-enabled capability	LLM-based intent classification, entity extraction, community report parsing, comparative summarization
R5	Routing / orchestration	Explicit deterministic router dispatches to Graph Engine, Retrieval Engine, or AI Engine; includes clarification prompts and out-of-scope fallback
R6	End-to-end interface	Gradio/Streamlit chat UI connected to FastAPI backend
R7	Reliability & safeguards	Input validation (geocoding check), ETA sanity check, confidence thresholds, out-of-scope rejection, no raw LLM passthrough
R8	Evaluation	50-case test benchmark, intent accuracy metric, ETA error metric, incident precision metric, documented improvement cycle

---

8. Evaluation Plan

8.1 Test Benchmark

A curated set of **50 test cases** across all intent categories:

Category	Count	Example
Route queries	20	"How to get from A to B by time T?"
Information retrieval	10	"Is road X flooded?"
Comparison queries	5	"Motorbike vs. bus to Y?"
Report submission	5	"Report: accident on Z street"
Out-of-scope	5	"Tell me a joke" / "What's the weather in Tokyo?"
Edge cases	5	Empty input, unknown location, conflicting constraints

8.2 Metrics

Metric	Measurement Method
Intent classification accuracy	Compare predicted intent vs. gold label on test set
Entity extraction F1	Token-level F1 for location, time, vehicle entities
ETA accuracy	$\lvert ETA_{predicted} - ETA_{actual} \rvert / ETA_{actual}$ averaged over sampled routes
System latency	Wall-clock time from user message to response
User satisfaction	Likert-scale survey on 5 volunteer testers

8.3 Planned Improvement Cycle

1. **Iteration 0 (Baseline):** Pure LLM classification + static-weight Dijkstra → measure all metrics.
2. **Iteration 1:** Add rule-based fast path for simple intents → expect intent accuracy improvement.
3. **Iteration 2:** Add dynamic time-dependent weights + weather overlay → expect ETA accuracy improvement.
4. **Iteration 3:** Add community incident retrieval → expect risk-aware routing improvement.

Each iteration's metrics will be recorded and compared — this directly addresses the rubric's demand for *"evidence of iterative CT application."*

---

9. Risk Analysis & Mitigations

Risk	Likelihood	Impact	Mitigation
LLM API rate limits / downtime	Medium	High	Cache frequent intents; rule-based fallback for common patterns
Inaccurate community reports (trolling)	Medium	Medium	Confidence scoring; require multiple corroborating reports; time-decay expiry
OSM data staleness	Low	Medium	Periodic graph refresh; manual correction for known road changes
Vietnamese NLU accuracy	High	Medium	Bilingual few-shot examples; fallback to English prompt translation
Scope creep	High	Medium	Strict non-goals definition; router rejects out-of-scope queries

---

10. Relationship to Course Skills (Lab Alignment)

Lab Week	Skill	How Used in UrbanPulse
1	Team formation, Git, AI policy	Project setup, repo, responsible AI logging
2	Flowcharts, Colab, Ollama	System flowchart design; LLM integration via Ollama
3	Gradio/Streamlit, CSV/JSON/JSONL	Chat UI; incident database in JSONL format
4	Proposal & PoC presentation	This proposal; PoC = basic intent → route pipeline
5	FastAPI, ngrok	Backend API serving the routing engine
6	OpenStreetMap API, OpenWeather API	Graph data extraction; weather-aware edge weights
7	Google Translate, Hugging Face	Vietnamese/English support; sentiment analysis on reports
8	Firebase	Persistent incident database storage
9	Unit testing, pytest	Benchmark test suite for all modules

Lab Week	Skill	How Used in UrbanPulse
10	Final presentation	Engineering defense of the full system

---

11. Summary

UrbanPulse transforms our group's routing algorithm project into a full *Smart Virtual Assistant* by wrapping the time-dependent graph routing engine inside a conversational pipeline. The key CT contributions are:

- **Decomposition**: Five cleanly separated modules, each independently testable.
- **Pattern Recognition**: Periodic congestion modeling, community report template matching, intent classification patterns.
- **Abstraction**: Layered interfaces hide data sources, AI models, and graph internals from each other.
- **Algorithm Design**: Hybrid deterministic + AI intent classification; context-aware A* with dynamic risk overlays; iterative improvement cycle with quantified benchmarks.

The system uses AI *where justified* (NLU, unstructured report parsing) and deterministic logic *where correctness matters* (routing, constraint validation, fallback handling) — exactly as the project specification demands.

"AI is one computational component, not the architecture." — Course Project Specification, 2026